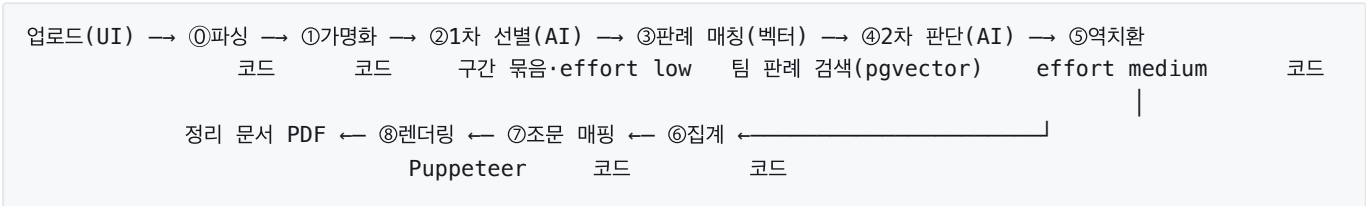


Safe Guard — 대화 분석·문서 생성 작동 방식

담당 기능 #2(수집한 대화의 AI 분석) · #4(분석 결과의 문서 정리) / NestJS 백엔드
마지막 갱신 2026-10-05 · main(판례 검색·회원)과 합쳤다. 저장소는 팀 Postgres로 옮겼고 판례 매칭은 팀 판례 검색 모듈에 연결됐다(3·4절). 실측 수치는 구간 단위 전환 후 실시간으로 갱신된다(5절).

1. 한눈에 보기

대화 내보내기 파일 하나가 정리 문서 PDF가 되기까지:



- AI 호출은 ②와 ④, 딱 두 번. 나머지는 전부 결정론적 코드다.
- 판단 단위는 "구간" — 같은 행위를 이루는 메시지 묶음(예: 심부름 요구 → 거절 무시 → 불이익 암시 → 요구 가중 = 구간 1개). 단독 발화도 구간 1개다. 법적 판단 단위가 개별 문장이 아니라 행위이기 때문이고, 공갈처럼 요건이 여러 메시지에 나뉘는 사안의 판례 매칭이 좋아진다.
- 원칙: 의미 판단은 AI가, 숫자 집계·법조항은 코드가. AI에게 법조를 맡기면 없는 조항을 지어내고(할루시네이션), 개별 사안에 법을 적용해 판단해 주는 것은 법률사무여서 변호사법 문제가 생긴다. 조문·형량은 국가법령정보 API에서 수집한 원문만 쓴다.
- 이 분석은 참고용이며 법적 판단이 아니다. 문서 매 페이지에 면책이 박힌다.

실측 (57건 학급 단톡 샘플, 2026-09-09):

단계	값
1차 선별	약 12~16초, 후보 구간 6개(메시지 31건)
2차 판단	약 15초, 확정 구간 6개 (탈락 0)
토큰	입력 약 10,800 / 출력 약 1,700 (메시지 단위 시절의 약 3분의 1)
서버 왕복	약 27초 (판례 미연결 상태)
정리 문서	A4 4페이지, 약 470KB (확정 구간 6개 기준)

2. 코드 지도

```

src/safeguard/
├─ kakao-parser.ts           ⑩ 카톡 내보내기 파서 (3형식 자동 감지)
├─ anonymizer.ts            ① 가명화 - 실명·연락처는 기기 밖으로 안 나감
├─ safeguard-analysis.service.ts ②④⑥ 1차·2차 프롬프트, 코드 패턴 검출, 집계
├─ precedent-provider.ts     ③ 판례 모듈 계약(입출력 바디) + Noop 구현
├─ vector-precedent-provider.ts ③ 팀 판례 검색(src/precedent)을 계약으로 감싼 어댑터
├─ law-registry.ts           ⑦ 유형→조문 매핑, 형량 추출, DM 분기
├─ law-api.service.ts        법령 API 동기화 (런타임 호출 안 함)
├─ report-pdf.service.ts     ⑧ Puppeteer PDF
├─ sessions.service.ts / .controller.ts 자료 업로드·조회·삭제 (/api/sessions)
├─ analyses.service.ts / .controller.ts 분석 실행·조회·목록·삭제·PDF (/api/analyses)
├─ storage/safeguard-store.ts 저장소 계약 (session · analysis · utterance)
├─ storage/postgres-safeguard-store.ts 운영 구현 (팀 Postgres, TypeORM DataSource)
├─ storage/sqlite-safeguard-store.ts 테스트 구현 (better-sqlite3, 메모리)
├─ dto/session.dto.ts · dto/analysis.dto.ts 요청 검증 (class-validator)
├─ safeguard.controller.ts   GET / (UI)
├─ safeguard-infra.module.ts 저장소·판례 매칭의 운영 구현을 정하는 곳 (e2e가 통째로 교체)
└─ safeguard.module.ts

```

src/migrations/20261005000000-create-safeguard-tables.ts safeguard_* 테이블 4개

data/law-articles.json 조문 16건 - 법령 API 원문 스냅샷
public/safeguard-ui.html 모바일 UI (세션 → 분석 → PDF 흐름)

3. HTTP API

경로·요청·응답 예시의 정보는 팀 노션 Endpoint DB다. 여기서는 구조와 구현상 결정만 적는다. 자원은 팀 데이터 모델을 따른다 — session(올린 자료 한 건) > analysis(분석 한 번)

utterance(의심 구간 하나).

Method	Path	하는 일
GET	/	테스트용 모바일 UI
POST	/api/sessions	내보내기 원문 업로드·파싱 → 세션 ID, 참여자 목록, 형식, 기간
GET	/api/sessions/{sessionId}	세션 메타(메시지 원문은 안 줌) + 이 세션을 쓴 분석 ID들
DELETE	/api/sessions/{sessionId}	삭제. 분석이 쓰고 있으면 409(analysisIds 동봉)
POST	/api/analyses	세션 ID + context + victimName → 2패스 실행, ndjson 스트리밍, 결과 저장
GET	/api/analyses?limit=&offset=	내 분석 목록(최신순). 응답은 { "analyses": [...] }
GET	/api/analyses/{analysisId}	결과: utterances(원문 발체 포함)·patterns·summary·precedents
DELETE	/api/analyses/{analysisId}	삭제(구간까지 함께)
GET	/api/analyses/{analysisId}/report?legal=	정리 문서 PDF. legal=false 면 법조·판례 없이 사실만

흐름

POST /api/sessions	파싱·참여자 집계 (AI 없음)	→ sessionId
POST /api/analyses	검증(404·400) → 분석 행 running → 스트리밍 시작	
	→ 엔진 2패스 → 구간 저장 → done	→ 마지막 줄 result
GET .../report	저장된 구간 + 세션 원문으로 PDF (매번 생성, 캐시 없음)	

- 검증은 스트림을 열기 전에 끝낸다. 없는 세션(404), 참여자에 없는 피해자(400), enum 밖 context(400)는 보통의 JSON 에러로 나간다. 스트림이 열린 뒤의 실패는 `{"type": "error", "analysisId", "message"}` 한 줄로 끝나고 분석 행은 `status: error` 로 남는다.
- utterance 1개 = 엔진의 flagged 항목 1개. `messageNos · harmTypes · severity · reason · appliedPrecedentIds` 를 그대로 저장하고, 응답 때 세션 원문에서 `messages` (번호·시각·발신자· 본문)를 붙여 준다. 클라이언트는 더 이상 파싱하지 않는다.
- `precedents` 는 매칭 후보 전체가 아니라 2차가 실제 근거로 쓴 사건번호만 남긴다.
- 지금은 분석 1건 = 세션 1개. `sessionIds` 는 배열이지만 2개 이상이면 400. 엔진이 대화 하나를 한 문맥(context)으로 보기 때문이고, 여러 자료 묶음은 세션별 실행 후 병합으로 확장한다.
- `context` 는 enum으로 강제 검증한다(ValidationPipe). 이 값 하나로 적용 법조가 통째로 걸린다.
- 에러 바디는 노션 명세대로 `{ "message": "한 문장" }` 이다. 전역 파이프는 `src/validation-pipe.ts` 의 `buildValidationPipe()` 하나이고(main.ts·e2e 공용), class-validator의 메시지 배열을 한 문자열로 합친다.
- `whitelist + forbidNonWhitelisted` (팀 main 설정)라서 검증 데코레이터가 없는 DTO 필드가 오면 400이다. 필드를 추가할 땐 반드시 데코레이터를 붙일 것.
- 인증은 아직 붙이지 않았다. 팀 회원 모듈(`src/users`, 쿠키 세션)이 main에 들어왔으므로 다음 단계는 가드를 붙이고 세션·분석에 소유자(`user_id`)를 기록하는 것이다.

저장소 — storage/

`SafeguardStore` 인터페이스(`safeguard-store.ts`) 뒤에 구현이 둘 있다.

- 운영: `Postgres` (`postgres-safeguard-store.ts`). 팀 DB를 그대로 쓴다(AppModule의 TypeORM DataSource). 테이블은 `safeguard_sessions · safeguard_analyses · safeguard_analysis_sessions` (조인) · `safeguard_utterances` 네 개이고 마이그레이션이 만든다(부팅 때 자동 실행). 로그인 세션(`user_sessions`)·판례 분석(`precedent_analyses`)과 헛갈리지 않게 접두사를 붙였다. API 경로는 그대로다.
- 테스트: `SQLite 메모리` (`sqlite-safeguard-store.ts`). e2e와 계약 테스트가 DB 없이 돈다.

배열·객체 필드는 `jsonb` 가 아니라 `json` 컬럼이다 — 조회 조건으로 쓰지 않고, `jsonb` 는 객체 키를 재정렬해 응답의 `typeCounts · precedents` 순서가 바뀐다. 두 구현은 같은 계약 테스트(`safeguard-store.spec.ts`)를 통과해야 하고, Postgres 쪽은 실제 DB가 있을 때만 돈다(6절). 어느 구현을 쓰지는 `safeguard-infra.module.ts` 의 `SAFEGUARD_STORE` 프로바이더가 정한다.

4. 단계별 상세

① 파싱 — kakao-parser.ts

카카오톡 내보내기 두 형식을 자동 감지한다:

PC형: [민준] [오후 2:20] 본문 ← 날짜는 구분선에서 상태로 유지
모바일형: 2026. 6. 23. 오후 10:59, 이름 : 본문 ← 매 줄에 날짜
2026. 6. 23. 오후 10:59: A님이 B님과 C님을 초대했습니다. ← 시스템 줄은 침표가 아니라 콜론
메시지가 삭제되었습니다. ← 시각 없는 단독 줄

처리하는 것: 오전·오후→24시간(오후 12시는 12시, 오전 12시는 00시), 여러 줄 메시지 병합, 시스템 이벤트(나감·초대·강퇴·삭제) 추출, 다중 초대 대상 분해 ("A님, B님과 C님을 초대" → targets: [A,B,C]).

시스템 이벤트가 중요한 이유: 방쪽은 텍스트에 욕설이 0건이라 입·퇴장 로그가 유일한 증거다.

① 가명화 — anonymizer.ts

외부(AI API·판례 검색)로 나가기 전에 신원을 지운다:

민준→학생A · 윤아→학생E ... "야 김윤아" → "야 학생E" (성이 붙은 형태까지 흡수)
전화번호 → [전화번호] · 주민번호 → [주민번호] · URL → [링크]

- 가명↔실명 매핑은 서버 메모리에만 존재한다. AI가 돌려준 reason 은 역치환한다.
- 메시지 번호(no)는 순서 기반이라 가명화와 무관하게 앵커로 동작한다.
- 한계: 참여자 목록에 없는 제3자 실명은 잡지 못한다. 완전한 익명화가 아니라 최선 노력(best-effort) 마스킹이다.

같은 시점에 코드 패턴 검출이 돈다 — AI 없이 결정론적으로:

방쪽: 피해자 초대·입장 후 10분 내 타인 2명 이상 퇴장

카톡감옥(퇴장 → 재초대 반복)은 패턴에서 뺏다(2026-09-10). 카톡은 방을 나가면 그 방 기록이 기기에서 지워져서, 피해자 본인의 내보내기에는 퇴장 이전 기록도 재초대 흔적도 남지 않는다. 방에 남아 있던 제3자의 내보내기에만 보이는데, 이 앱은 본인 업로드가 전제다.

의미 판단이 필요한 나머지 패턴(떼카·서툼·은따·저격글·반톡배제)은 1차 AI가 관찰하고, 최종 patterns 는 둘의 합집합이다. AI가 놓쳐도 코드가 잡는 안전망.

② 1차 선별 — AI, effort low

번호 매긴 대화 전문을 주고 "유해 후보를 골라 같은 행위끼리 구간으로 묶어라"를 시킨다. 1차는 전체 대화를 읽는 유일한 AI 단계다.

- 애매하면 포함 — 여기서 빠진 메시지는 이후 단계에 전달되지 않는다(미탐은 복구 불가). 잘못 들어간 것은 2차가 거른다.
- 확실히 무해한 것만 제외: 일상 대화, 피해자 발화, 무례·불쾌 수준, 지목 없는 감탄사형 욕설("ㅂㅂ 시험 망함"), 일상 초성체(ㅇㅋ·ㄱㅈ·ㄴㅇㄹ...), 쌍방 장난.
- 동조성 반응은 포함: 비하 직후의 "ㅋㅋㅋㅋ"."ㅇㅈ"."ㄱㄱ"는 집단따돌림의 구성 발화다(가해자만 웃고 대상자는 웃지 않으면 조롱).
- 구간 묶기 규칙: 구간 하나 = 행위 하나 (며칠에 걸쳐도 같은 요구의 반복이면 한 구간), 다른 행위는 다른 구간, messageNos 에는 가해 측 발화만(피해자 발화는 흐름 판단에만 사용), 동조성 반응은 따라붙은 발화와 같은 구간, 한 메시지는 한 구간에만.
- 구간마다 세 가지를 붙인다: - messageNos — 구간에 속한 가해 발화 번호 (오름차순) - victimIdentifiable — 특정성. 이름·2인칭·멘션뿐 아니라 "어떤 애가" 식 맥락 지목 포함 - summary — 누가 무엇을 어떤 순서로 했는지 사실만 한두 문장. 2차의 참고이자 벡터 검색 질의로도 쓰인다. 죄명·유형 단어 금지 (1차의 판정이 섞이면 2차가 그것에 끌려가는 앵커링이 실측에서 확인되어 금지 규칙을 넣었다)

- 코드가 AI 출력의 번호를 정리한다 — 범위 밖·중복 제거, 오름차순, 빈 구간 제거.

③ 판례 매칭 — 팀 판례 검색 모듈 (어댑터로 연결)

계약(입출력 바디)은 `precedent-provider.ts` 에 타입으로 고정돼 있다:

```
입력:  analysisTargets[] = { targetMessageId, targetText,
                             legalContext: { isGroupChat, audienceCount, victimIdentifiable } }
출력:  precedentDict      = { 사건번호: 요지 한 줄 }          ← 중복 제거된 사전
      analysisTargets[] + matchedPrecedentIds[3]             ← 구간엔 번호만
```

타킷 1개 = 구간 1개. 계약 형태는 메시지 단위 때와 동일하고 의미만 바뀌었다 —

`targetMessageId` 는 구간의 메시지 번호 나열("32,34,35,40"), `targetText` 는 1차의 사실 요약 + 구간 원문을 이어 붙인 문자열이다. 요약은 구어 원문보다 판시사항 문체에 가까워 검색 질의로 유리하고, 원문은 검색 근거의 손실을 막는다. 벡터 모듈 쪽은 "텍스트가 여러 문장일 수 있다"는 것만 알면 되고 코드 수정은 필요 없다.

- `isGroupChat` · `audienceCount` 는 코드가(방 정보), `victimIdentifiable` 은 1차 AI가 채운다.
- 운영 구현은 `vector-precedent-provider.ts` — 팀 판례 검색 모듈(`src/precedent` 의 `PrecedentMatchService` , pgvector + OpenAI 임베딩)을 같은 프로세스에서 불러 우리 계약으로 바꾼다. 엔진은 `PRECEDENT_PROVIDER` 토큰으로 주입받고, 구현은 `safeguard-infra.module.ts` 가 정한다.
- 어댑터가 맞추는 것: 검색 모듈은 호출당 구간 10개·5000자까지라 나눠 호출하고 자른다. `targetMessageId` 는 보내지 않고 응답 순서로 맞춘다. **사건번호가 없는 판례는 뺀다**(우리 ID가 사건번호다).
- **사건의 요지는 판례 API 원문이다**. 검색 응답의 `searchSummary` · `factPattern` 은 AI가 쓴 문장이라 쓰지 않고, 저장된 판시사항(없으면 판결요지) 원문의 첫 항목을 200자까지 넣는다 (지어낸 요지가 들어가면 2차 판단 전체가 오염된다). 원문을 못 구한 판례는 뺀다.
- top-3는 임계값이 없어서 무관한 판례도 붙는다 — 그래서 2차 프롬프트에 "무관하면 버려라"가 있다. 검색 응답에 점수가 있으므로 하한선을 두려면 어댑터에서 건다(지금은 두지 않았다).
- **검색이 실패해도 분석은 계속된다**. 임베딩 키(`SECRET_KEY`)가 없거나 판례 DB가 비어 있으면 경고 로그만 남기고 판례 없이 2차로 넘어간다(`match_done` 의 `precedents: 0`).
- 2차가 낸 `appliedPrecedentIds` 중 사전에 없는 번호는 코드가 걸어낸다(경고 로그).

프로토타입 폴더의 `판례-풀.json` 14건은 회귀 러너(`pipeline.mjs`)의 목업 매칭에만 쓰인다. 실서비스의 판례는 팀 DB에서 오므로 러너와 판례 적용 결과가 다를 수 있다(7절).

④ 2차 판단 — AI, effort medium

대화 전문 없이 **판례 사전 + 후보 구간 목록**을 받는다. 구간마다 첫 발화 앞 1줄부터 마지막 발화 뒤 1줄까지의 원문이 통째로 들어간다:

[구간 0032,0034,0035,0040] 피해자 식별: 예 · 판례 후보: 2003도709, ...

[0031] 학생A: 야 학생E

▶ [0032] 학생A: 내일 매점에서 빵 사와라

← ▶ = 구간 소속 발화

[0033] 학생E: 나 돈 없는데

← 표시 없음 = 사이의 피해자 응답 (맥락)

▶ [0034] 학생A: 없으면 만들어와야지

▶ [0035] 학생A: 안사오면 어떻게 되는지 알지

...

▶ [0040] 학생A: 그럼 내일은 두 개 사와

[0041] 학생E: 알겠어

요약: 심부름을 요구하고 거절을 무시한 뒤 불이익을 암시, 이튿날 요구를 늘림 ← 1차 요약 (참고용)

2차는 구간 단위로 유형·정도를 판정하고, 한 구간이 실제로 별개 행위 둘이면 messageNos 를 나눠 두 항목으로 낼 수 있다 (합치기는 금지). 유해하지 않은 발화 번호는 빼고 반환한다.

핵심 규칙:

- 판례는 유사도 순 후보일 뿐 적용 보증이 아니다. 무관하면 버리고, 실제 근거로 쓴 것만 appliedPrecedentIds 에 넣는다. 제공된 사건번호 외는 지어내지 못한다.
- 유해성이 인정되지 않는 후보는 반환하지 않는다 — 1차의 "넉넉히 포함"과 짝.
- 성립 요건 판단이 전부 여기 있다: - 협박: 구체적 해악의 고지가 있어야 한다. 압박·재촉·조롱·퇴장 금지만으로는 협박이 아니다. 말다툼 중 즉발적 분노 표시는 제외하되, 거부 의사를 밝힌 상대에 대한 일방적 해악 고지는 협박이다. - 명예훼손: 검증 가능한 구체적 사실의 적시 필요. 추상적 경멸은 모욕. - 공연성: legalContext 기준. 1:1이면 모욕·명예훼손 성립이 어렵고, 전파 가능성 예외는 판례가 있으면 그에 따른다. - 모욕과 언어폭력을 같은 메시지에 동시에 붙이지 않는다(중복 조문 방지).
- 갈취·강요: 요구와 해악 고지가 한 구간 안에서 결합돼 있으면 그 결합 자체를 평가한다 — 메시지 단위 때는 요건이 조각나서 놓치던 부분이다.
- 유형 8종: 언어폭력·명예훼손·모욕·성희롱·협박·스토킹·집단따돌림·갈취강요. 구조화 출력(zod 스키마 강제)이라 이 밖의 값은 물리적으로 나올 수 없다.

⑤⑥ 역치환·집계 — 코드

- AI가 쓴 reason 의 가명을 실명으로 되돌린다.
- 집계는 코드가 센다(LLM은 숫자를 틀린다). 건수·유형·정도는 구간 단위, 발신자·날짜는 구간에 속한 메시지 전부로 센다 (구간 하나에 발신자가 둘이면 둘 다 집계됨 — 동조자 포함). 기간, 반복성은 메시지 개수가 아니라 서로 다른 날짜 수로 판단한다 (같은 날 몇 시간 내 연속 전송은 한 덩어리 — 대법원 2023도5814 취지). 서로 다른 3일 이상이면 반복성 인정 범위.
- 발신자별 집계는 학폭법상 따돌림의 "가해자 2명 이상" 요건 판단 재료가 된다. 동조 발화("ㅋㅋ", "ㅇㄹ")가 잡히면 동조자도 별도로 집계된다.

⑦ 조문 매핑 — 코드 (law-registry.ts)

AI가 낸 harmTypes 를 조문으로 바꾼다. AI는 여기 관여하지 않는다.

협박 → 형법 제283조 — ① 3년 이하의 징역, 500만원 이하 벌금

· ② 5년 이하의 징역, 700만원 이하 벌금 [시행 2026.09.13]

반의사불벌죄 — 합의하면 처벌할 수 없습니다.

- 형량은 하드코딩이 아니라 data/law-articles.json 의 조문 원문에서 항 단위로 추출한다. 법이 개정되면 조문만 다시 받으면 자동 반영된다. (이 구조 덕에 개정으로 어긋난 형량 2건을 실제로 잡았다 — 정통망법 70조② 5천→7천만원, 저작권법 136조① 5년/5천→7년/1억)

- **1:1 대화 분기**: 공연성이 필요한 유형(모욕·명예훼손)은 `context === 'dm'` 이면 자동으로 정통방법 74조(반복 전송)로 전환된다. 성립 불가능한 죄명을 안내하면 고소 각하·역고소 위험이 있기 때문.
- 소추요건(친고죄 6개월·반의사불벌·합의 무관)도 유형별로 함께 붙는다.
- 조문 동기화: `law-api.service.ts` 가 국가법령정보 API로 주기 실행용. **런타임에는 절대 호출하지 않는다** (법은 몇 달 단위로 바뀌고, 외부 API 장애가 분석 실패로 전파되면 안 된다).

⑧ PDF — `report-pdf.service.ts`

Puppeteer(헤드리스 크롬)로 A4 렌더. 브라우저 인스턴스는 재사용하고 모듈 종료 시 정리.

- 항목 1개 = 구간 1개. 구간이 여러 발화면 발화마다 시각·발신자를 붙여 한 줄씩 인용한다.
- `legal: true` — 항목마다 관련 조항·형량·소추요건, 말미에 삭제·차단 절차 (정통방법 44조의2/44조의7), 학폭 절차, 민사 트랙(민법 750·751·755).
- `legal: false` — 조항 대신 AI의 판단 근거(`reason`)만.
- AI가 쓴 `reason` 과 판례 사건번호는 `legal: true` 문서에 **넣지 않는다** — 수사·상담 기관이 보는 자료에 AI 추론이 사실처럼 섞이면 안 되고, 판례 적용을 우리가 보증하는 모양이 되면 안 된다.
- 항목은 `break-inside: avoid` 로 페이지 경계에서 잘리지 않는다. 매 페이지 푸터에 "본 문서는 참고용이며 법률 자문이 아닙니다".

5. 검증 상태

구간 단위 전환(2026-09-08) 다음 날 실AI로 다시 잼다. 구간 로직은 단위 테스트 4개 (타킷 변환·집계·문서·패턴)와 e2e로도 잡는다.

프로토타입 폴더(예소)의 회귀 러너(`eval-runner.mjs`)로 성격이 다른 3개 대화를 자동 검사한다 — 프롬프트·풀·코드를 고치면 반드시 다시 돌릴 것. 2회 연속 3/3 통과:

케이스	결과
중증 학급 단독 57건	후보 구간 6 → 확정 6 · 즉시조치 9건 미탐 0 · 해악 고지 없는 구간에 협박 안 붙음
정상+쌍방장난 18건	후보 구간 0 — 1차에서 끝나 2차 호출 없음 (약 3초, 토큰 3,700)
1:1 협박 12건	후보 구간 2~3 → 확정 2 · 사진 유포 암시와 금품 요구가 한 구간으로 묶여 갈취강요·협박

확인 포인트였던 셔틀 시퀀스: 뺨 요구(32)부터 "두 개 사와"(40)와 동조 웃음(42·43)까지 8건이 구간 하나로 묶였고, 2차가 갈취강요·협박·집단따돌림으로 판정하며 공갈 판례 2003도709를 적용했다. 메시지 단위 시절에는 같은 행위가 5건으로 흩어져 따로 판정됐다.

실측에서 드러난 것:

- 2차가 정의 밖 `harmType`을 낸 적이 1회 있었다(2차 호출 4회 중 1회). SDK의 `parse()` 는 값 하나 때문에 통째로 실패하므로 직접 파싱해 enum 밖 값만 건어내도록 바꿨다(`callStructured`). 건어낸 값은 경고 로그로 남는다.
- 구간 묶임은 실행마다 조금 다르다(같은 대화에서 30~31건, 저격률 패턴 유무). 잡히는 번호 집합은 안정적이지만 구간 경계는 흔들릴 수 있어, 골든셋은 "번호가 어느 구간에도 속하는가"로 본다.
- 1차 요약문을 역치환하면 조사가 어긋난 채("지훈가") 문서에 나올 수 있다(7절).

NestJS 서버 E2E(2026-09-10, 세션·분석 자원 구조 + SQLite, 판례 미연결): 세션 생성(57건·참여 5명) → 분석 31초·확정 구간 8 → 조회·목록 → 세션 삭제 409 → PDF 479KB(legal=false 349KB) → 잘못된 context 400·없는 분석 404. 이 실행에서 2차가 enum 밖 유형("강요")을 한 번 더 냈고

callStructured 가 건너뛴다.

main 병합 후 E2E(2026-10-05, Postgres 저장 + 판례 어댑터, 임베딩 키 없음): 마이그레이션 4개 적용 → 세션 생성 → 분석 25초·후보 6·확정 6(판례 검색은 키가 없어 경고 후 판례 없이 진행) → 조회(스트림 결과와 동일)·목록 → 세션 삭제 409 → PDF 475KB(legal=false 345KB) → 삭제 204. 팀 라우트 (/precedent/... /users/...)와 한 서버에서 공존한다. **실제 판례가 2차에 들어간 실행은 아직 없다** — 판례 DB 데이터와 임베딩 키가 있는 환경에서 재야 한다.

자동 테스트(우리 모듈): 단위 26개 — 파서 6·엔진 5·판례 어댑터 5·저장소 계약 5×2(SQLite는 항상, Postgres는 DB가 있을 때) — 와 e2e 6개(엔진·PDF 생성기는 스텝, 저장은 메모리 SQLite).

6. 실행·운영

```
npm install
docker compose up -d db      # Postgres(pgvector). .env 의 DB_PASSWORD 를 쓴다
npm run build
node dist/main.js            # http://localhost:3000 — 부팅 때 마이그레이션 자동 실행
```

- .env 에 DB_PASSWORD 가 없으면 앱이 뜨지 않는다. DB_HOST · DB_PORT · DB_USERNAME · DB_NAME 은 기본값(localhost·5432·postgres·hungry)이 있다.
- 판례 검색에는 SECRET_KEY (OpenAI 키 — 구간 텍스트 임베딩)와 판례 DB 데이터가 필요하다. 없으면 분석은 판례 없이 돈다. 임베딩으로 나가는 텍스트도 가명화된 것이다.
- 저장소 계약 테스트를 Postgres로도 돌리려면: SAFEGUARD_TEST_PG_URL=postgres://postgres:<pw>@localhost:5432/hungry npm test (safeguard_test 스키마를 만들었다 지운다. 변수가 없으면 Postgres 쪽은 skip).
- .env 에 CLAUDE_AI_API_KEY=sk-ant-... (main.ts가 ANTHROPIC_API_KEY로 옮겨 SDK에 전달). .gitignore 에 등록돼 있어 커밋되지 않는다 — 키를 코드나 커밋에 넣지 말 것.
- 법령 API용 API_KEY (open.law.go.kr OC 값)는 동기화 스크립트에만 필요.
- 분석 1회 = Claude API 2회 호출. 대화 크기에 따라 30초~2분, 토큰 1만~2만.
- safeguard_sessions.messages 에는 개인 대화 원문이 들어간다. DB 덤프를 공유하거나 커밋하지 말 것.
- better-sqlite3 (테스트용)는 네이티브 모듈이다. npm install 때 Node 버전에 맞는 바이너리를 받고, 실패하면 빌드 도구(python3·C++ 컴파일러)가 필요하다.
- 로그: 분석 시작/완료(분석 ID·건수·패턴·소요시간)·문서 생성(크기)이 AnalysesService 로거로 남는다.

7. 알려진 한계 / 남은 일

- 분석 1건 = 세션 1개 — 여러 자료 묶음 분석은 미지원(400). 세션별 실행 후 병합이 다음 단계.
- 인증 없음 — 팀 회원 모듈이 들어왔다. 가드를 붙이고 safeguard_sessions · safeguard_analyses 에 소유자 컬럼 (user_id)을 추가하는 마이그레이션이 다음 단계. 탈퇴 시 대화 원문을 지울지(CASCADE)도 그때 정한다.
- 구간 제외(PATCH utterances)·재분석 없음 — excluded 컬럼만 준비돼 있다.

- 판례가 들어간 2차 판단은 미검증 — 연결은 됐지만 실제 벡터 검색 결과로 2차를 돌려 본 적이 없다. 과적용·무관 판례 미탈락이 보이면 프롬프트나 어댑터의 점수 하한선을 손본다.
- 회귀 러너는 목업 판례로 돈다 — `eval-runner.mjs` 는 판례 풀 14건 + 문자 유사도라 실서비스와 판례 적용이 다를 수 있다. 미탐 검사는 판례와 무관해 유효하다.
- 카드 매핑(팀원, PR #3) — analysis에 카드 ID를 넣는 구조라 그때 필드와 저장소 계약을 같이 맞춘다.
- 대용량 청킹 없음 — 단일 패스라 수천 건 대화는 출력 한도를 넘을 수 있다. 세션(날짜) 단위 분할이 반복성 법리와도 맞는 방향.
- 리눅스 한글 폰트 — PDF 폰트가 macOS/Windows 기준이라 리눅스 배포 시 `fonts-noto-cjk` 설치 또는 폰트 임베드 필요.
- 역치환 시 조사 어긋남("민준가") — 사소하지만 `legal:false` 문서에 보일 수 있음.
- 유해 판정 기준의 근거(판례 조사)는 예소/safeguard-law-reference.md, 판단이 갈린 실측 기록은 예소/샘플-분석결과-AI.json 참고.